

Session 03

Production Model Training

Day 1 • 11:00 AM–12:00 PM • 60 minutes

Today's idea

A model is not production-ready just because it predicts accurately.

Focus

Reproducible training: config → data → features → train → evaluate → artifacts

Outcome

Students can run, inspect, modify and explain the pipeline.

Learning objectives

By the end of 60 minutes, learners should be able to...

Explain why configuration, logging and artifacts matter for reproducibility.

Read a compact training pipeline and map each script to a production responsibility.

Run the end-to-end CLI and identify the model, metrics and log outputs.

Change a configuration value and reason about the effect on the experiment.

Explain where preprocessing belongs and why it must be consistent between training and inference.

Notebook thinking vs production thinking

The goal is not to stop using notebooks — it is to move repeatable logic into testable code.

Notebook-first workflow

Cells run in an order chosen by the user.

State can remain hidden in memory.

Parameters may be changed manually.

Results may be copied into a report.

Good for exploration and teaching.

Production-style workflow

One command defines the run.

Config records important parameters.

Functions/modules have clear responsibilities.

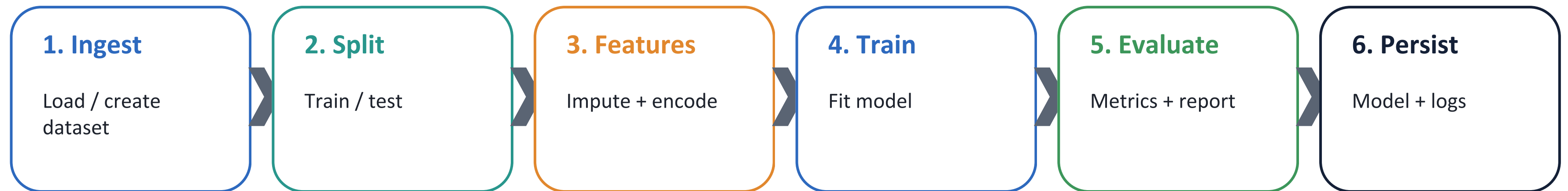
Artifacts are saved automatically.

Logs explain what happened.

Good for repeatability and handover.

The production training pipeline

Think of training as a controlled data-to-artifact transformation.



Key principle: each step should be understandable, repeatable and independently testable.

Folder architecture

Small modules make responsibilities visible.

main.py

Orchestrates the run
CLI + logging

data_loader.py

Dataset creation/loading
Train/test split

features.py

Imputation
One-hot encoding

train.py

Model selection
Fit + save

evaluate.py

Metrics
JSON report

config.yaml

Data + model + paths
Experiment settings

Configuration is the experiment record

Instead of changing code for every experiment, change controlled parameters.

```
data:  
  path: data/churn.csv  
  target_column: churn  
  test_size: 0.20  
  random_state: 42  
  
model:  
  type: logistic_regression  
  max_iter: 1000  
  
paths:  
  model_path: artifacts/model.joblib  
  report_path: artifacts/metrics.json  
  log_path: artifacts/training.log
```

Teaching question

Reproducibility

Separation

Why random_state matters

Same code + same data + same seed → comparable experiments.

Run A

random_state = 42
Same split
Same transformation
Comparable result

Run B

random_state = 42
Same split
Same transformation
Comparable result

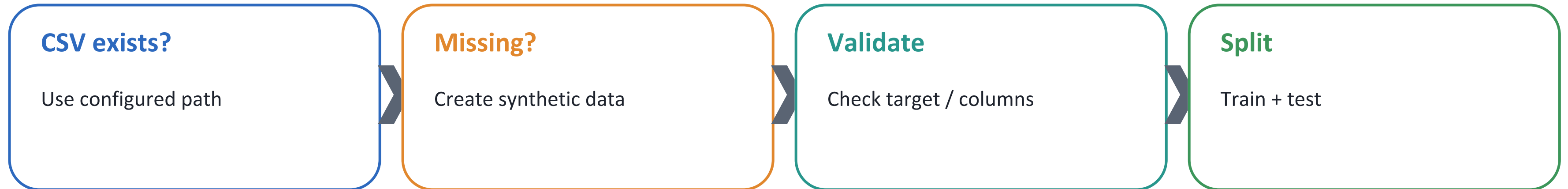
Run C

random_state = 7
Different split
Potentially different result

Important: a seed does not make a model universally better. It makes an experiment repeatable.

Data loading & splitting

The loader owns the dataset boundary — not the model.

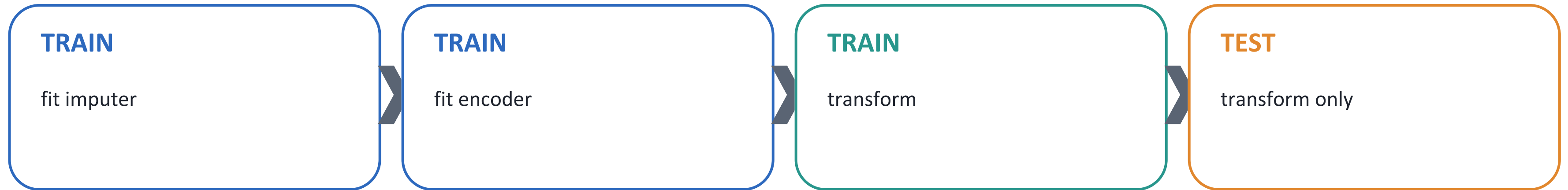


```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=config['data']['test_size'],  
    random_state=config['data']['random_state'],  
    stratify=y  
)
```

Teaching point

Feature engineering: fit on train, transform both

This is a critical leakage-prevention rule.



Correct: learn preprocessing parameters from training data only.

Why? The test set must remain an unseen estimate of generalization.

Bad pattern

`fit_transform(X)` before train/test split

Better pattern

`fit_transform(X_train); transform(X_test)`

Training: encapsulate model selection and fitting

train.py should make the model step easy to test and replace.

```
def train_model(X_train, y_train, config):  
    model = LogisticRegression(  
        max_iter=config['model']['max_iter']  
    )  
    model.fit(X_train, y_train)  
    return model  
  
model = train_model(X_train, y_train, config)
```

Design goal

Easy extension

Easy testing

Evaluation: accuracy is only one view

For classification, inspect several metrics and the confusion matrix.

Accuracy

Overall fraction predicted correctly.
Useful, but can hide class imbalance.

Precision

Of predicted positives, how many were actually positive?

Recall

Of actual positives, how many did we find?

F1

Balances precision and recall using their harmonic mean.

```
metrics = {  
    'accuracy': accuracy_score(y_test, y_pred),  
    'precision': precision_score(y_test, y_pred),  
    'recall': recall_score(y_test, y_pred),  
    'f1': f1_score(y_test, y_pred)  
}
```

Artifacts are the outputs of a training run

If you cannot inspect what happened, reproducibility is incomplete.

model.joblib

Serialized trained model.
Can be loaded later for inference.

metrics.json

Accuracy, precision, recall, F1
+ full classification report.

training.log

INFO-level record of the run:
what loaded, trained and saved.

Mental model: Code produces artifacts; artifacts provide evidence of what the code produced.

Run the pipeline

Start from the repository root so relative paths behave consistently.

```
python -m venv .venv

# Windows
.\.venv\Scripts\activate

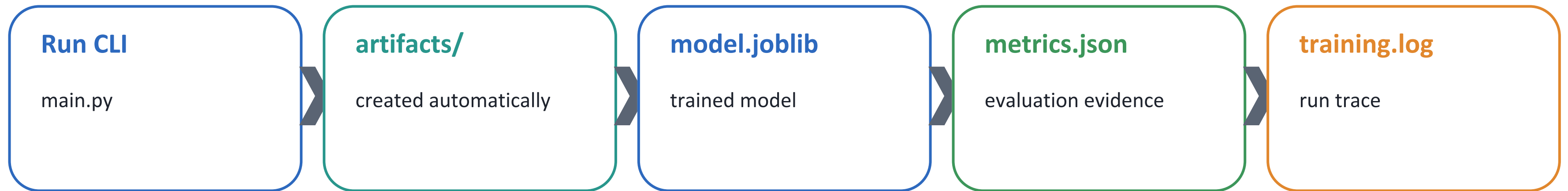
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt

python day1/03_production_training/scripts/main.py \
  --config day1/03_production_training/scripts/config.yaml
```


What should appear after the run?

Have students inspect the filesystem, not just the terminal output.



```
artifacts/  
├── model.joblib  
├── metrics.json  
└── training.log
```

Live demo: change one variable

Use configuration to demonstrate an experiment rather than editing the training code.

Experiment 1

test_size: 0.20
random_state: 42
Run → record metrics

Experiment 2

test_size: 0.30
random_state: 42
Run → record metrics

Discuss

Did metrics change?
Why might they change?
Was the comparison fair?

Teaching insight: controlled changes turn “I ran it again” into an experiment.

Optional extension: add RandomForest

A good production design makes model replacement local rather than a rewrite.

```
model:
  type: random_forest
  n_estimators: 200
  random_state: 42

# train.py
if model_type == 'logistic_regression':
    model = LogisticRegression(...)
elif model_type == 'random_forest':
    model = RandomForestClassifier(...)
```

Student task

Troubleshooting checklist

Teach students to diagnose systematically.

ModuleNotFoundError

Check the virtual environment.
Run `pip install -r requirements.txt`.

Target not found

Open the CSV.
Compare its headers with `data.target_column`.

No artifacts

Check paths in `config.yaml`.
Check the log for the failing step.

Unexpected metrics

Confirm `random_state`.
Inspect train/test sizes.
Check preprocessing.

Path problems

Run from repository root.
Prefer config-driven paths.

Logging

Open `training.log`.
Find the first ERROR, not just the final symptom.

Production checklist

Before calling a training script “production-style,” ask these questions.



Can another person run it from a clean environment?



Are important parameters stored in configuration?



Is the train/test split reproducible?



Is preprocessing fitted only on training data?



Are evaluation metrics saved as an artifact?



Is the trained model persisted?



Can logs explain what happened during the run?



Can one component be changed without rewriting the entire pipeline?

60-minute teaching plan

Keep the session hands-on: explain → run → inspect → modify → discuss.

0–10 min

Architecture
Why production-
style?

10–20 min

Read files
Map
responsibilities

20–35 min

Run pipeline
Inspect artifacts

35–48 min

Change config
Compare runs

48–57 min

Optional RF
Student extension

57–60 min

Recap
Exit questions

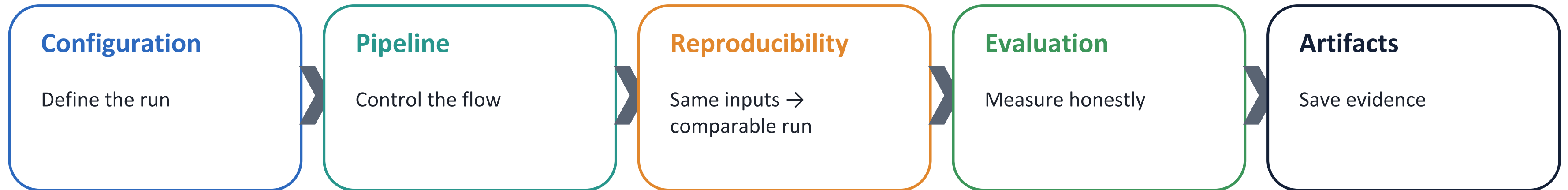
Exit questions

Use these to test understanding rather than memorization.

1. Why should test data not be used to fit an imputer or encoder?
2. What does `random_state` give us — and what does it NOT guarantee?
3. If metrics changed after changing `test_size`, does that automatically mean the model improved?
4. What is the purpose of `model.joblib`, `metrics.json` and `training.log`?
5. Which file would you modify to add a new model type?
6. If you had to hand this pipeline to another engineer, what would you want them to be able to reproduce?

Key takeaway

Production model training is a system, not a single `.fit()` call.



A production-style pipeline makes training repeatable, inspectable, and easier to change safely.